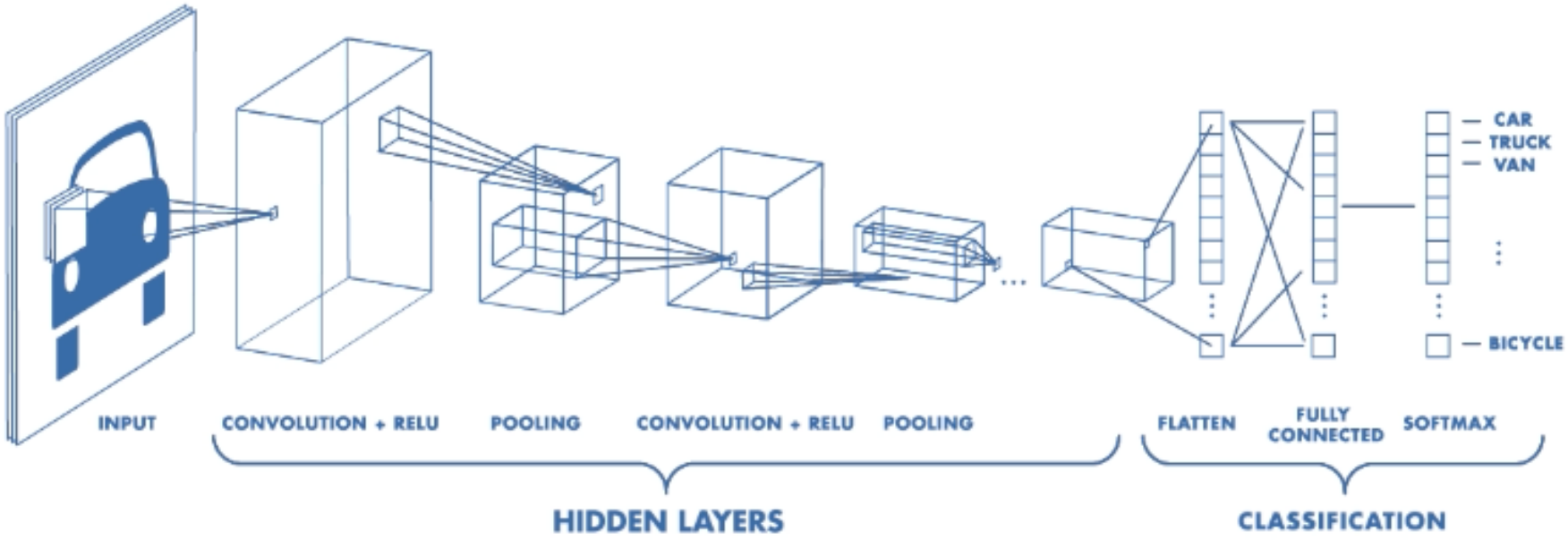
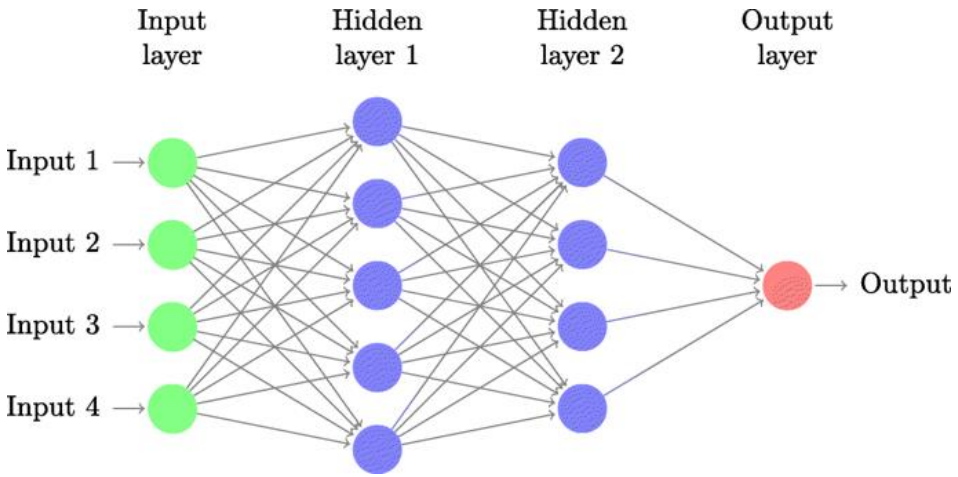
CONVOLUTIONAL NEURAL NETWORKS (Conv Net)

JNTUH DILT , AIML Session 5, 15 June 2024

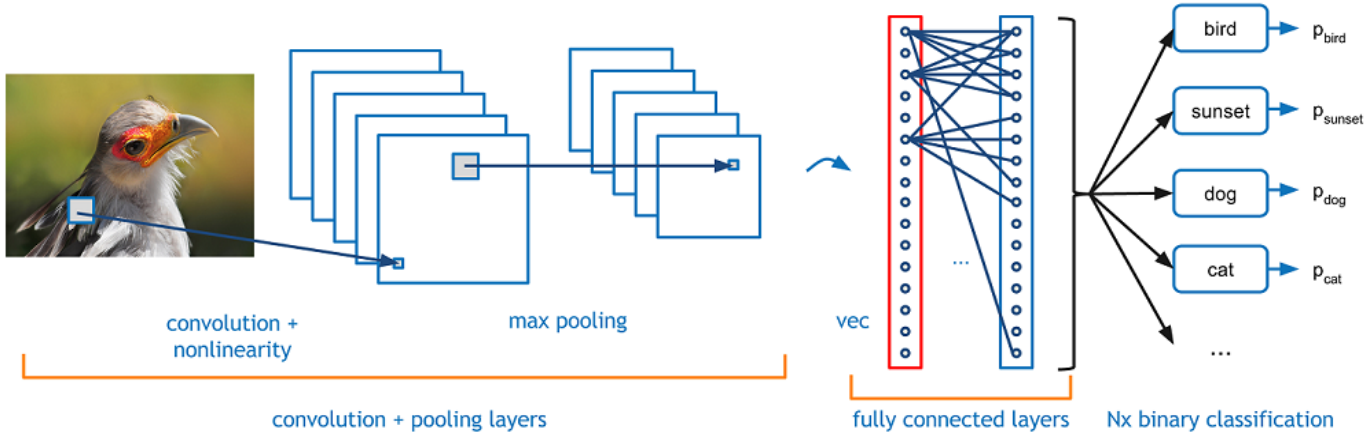


Architectures

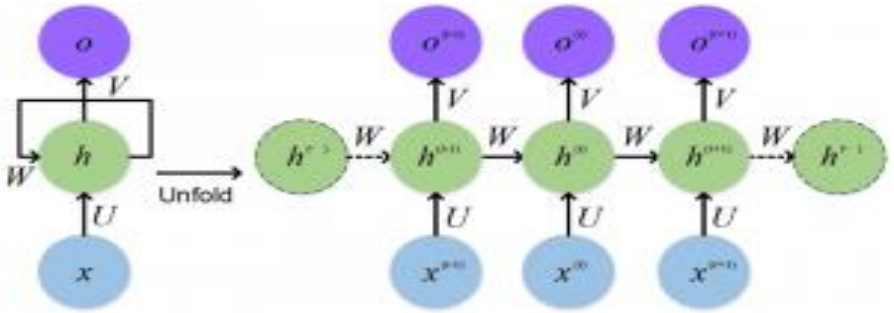
Artificial Neural Network(ANN)



Convolution Neural Network(CNN)

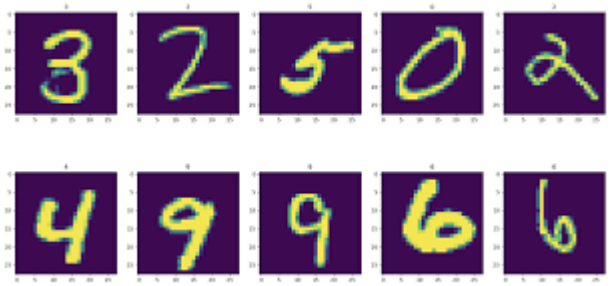


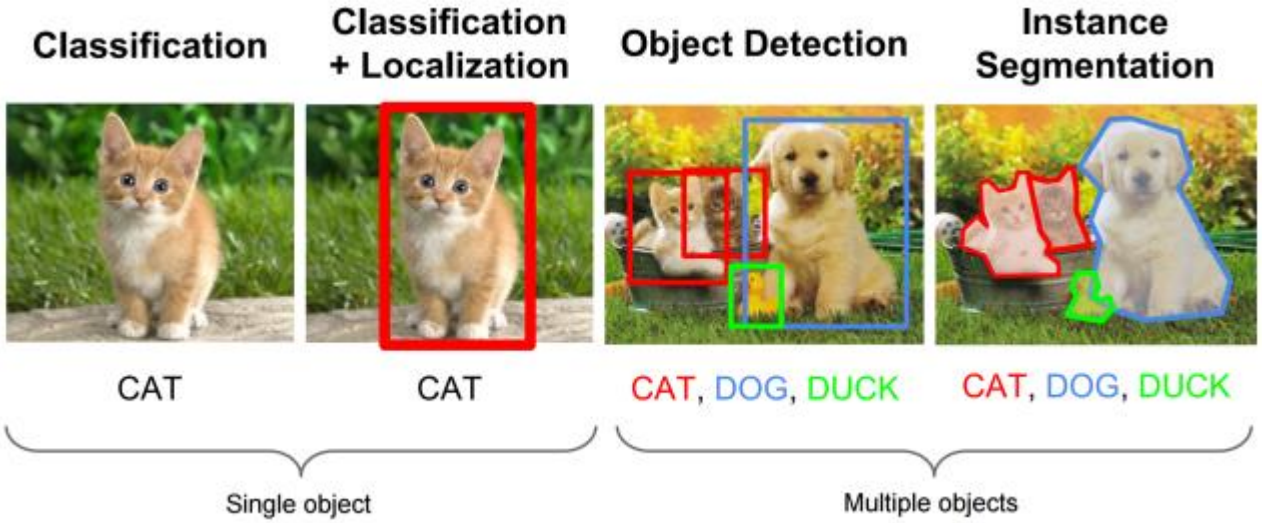
Recurrent Neural Network (RNN)

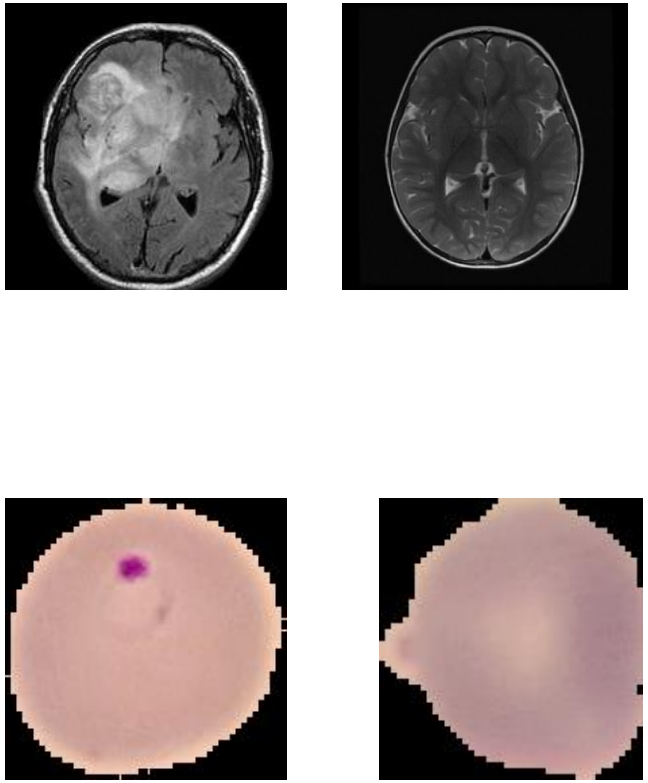


Applications:

- Image Classification
- Image Recognition
- Document Classification
- Medical Image Analysis

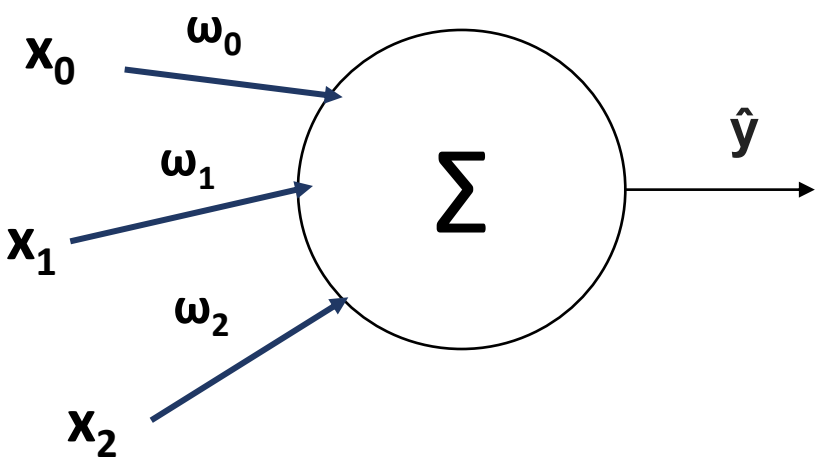






Neural Networks

linearity

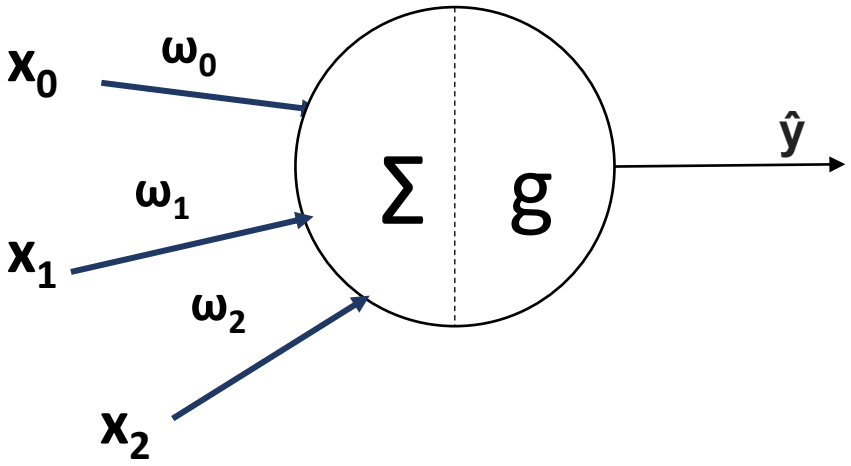


$$z = \sum_{i=0}^{n} \omega_i x_i$$

$$= \omega_0 + \omega_1 x_1 + \omega_2 x_2 .....$$

where **z is called summation function**

non-linearity



$$g(z) = \frac{1}{1 + e^{-z}} \ \varepsilon[0,1]$$

where **g(z) is called activation function** ,which is a **sigmoid** function.

If the **all the values** of the activation function **belong to a particular range,** then it comes under non-linearity.

Neural Networks

linear/summation function

$z = \sum_{i=0}^{n} \omega_i x_i$

$= \omega_0 + \omega_1 x_1 + \omega_2 x_2 \ldots\ldots$

here bias is ω_0

non-linear/activation functions

1. **sigmoid** : $g(z) = \frac{1}{1+e^{-z}} \; \varepsilon[0,1]$

2. **tanh** : $\tanh z = \frac{e^{z}-e^{-z}}{e^{z}+e^{-z}} \varepsilon[-1,1]$

3. **ReLU**: $f(z)=\max(0,z)$

4. $softmax(z_i) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \varepsilon[0,1]$

$$\sum_{i=0}^{n} softmax(z_i) = 1$$

where i = 1,2,3,…,n and

n is number of classes

Computer Vision

Some of the computer vision problems are:

1. Image classification
2. Object detection
3. Neural style transfer

One major problem with computer vision problems is that the input data can get really big. Suppose an image is of the size 68 X 68 X 3. The input feature dimension then becomes 12,288. This will be even bigger if we have larger images (say, of size 720 X 720 X 3). Now, if we pass such a big input to a neural network, the number of parameters will swell up to a HUGE number (depending on the number of hidden layers and hidden units). This will result in more computational and memory requirements – not something most of us can deal with.

Edge detection

- **Edge Detection** is simply a case of trying to find the regions in an image where we have a sharp change in intensity or a sharp change in color, a high value indicates a steep change, and a low value indicates a shallow change.

Convolutional Neural Networks

Types of Layers :

- Convolution layer
- Pooling layer
- Flattening layer
- Fully connected layer

1. Convolution layer (Convolution operation)

Types of Convolution:

- **I-Dimensional Convolution**

  input: text

- **2-Dimensional Convolution**

  input: gray-scale Image

- **3-Dimensional Convolution**

  input: color Image

Convolution Layer

Convolution operation: hyper parameters

1. **Filter/Kernel :** it is a weight matrix

1	0	-1
1	0	-1
1	0	-1

Vertical

1	1	1
0	0	0
-1	-1	-1

Horizontal

1	0	-1
2	0	-2
1	0	-1

Sobel filter

3	0	-3
10	0	-10
3	0	-3

Scharr filter

2. Stride: It is the **number of steps (pixels)** **the filter is to be moved** **horizontally or vertically over the input matrix**

when **stride is 1,** then we **move the filter to 1 step (pixel)** at a time

when **stride is 2,** then we **move the filter to 2 steps (pixels)** at a time etc

3. **Padding:** It is the process of **adding borders with zeros** to input matrix

4. **No.of filters:** for 3D input

Convolution Layer

Convolution Operation: Hyper Parameters

I) Filter/Kernel matrix (Weights matrix)

II) Stride – 1,2,3,4,…
(default stride is 1)

III) Padding – 0,1,2,3,...
(default padding is 0)

IV) No.of filters/No.of Kernels (for 3D input)

For 1-Dimensional (signal): Parameters I,II are used

For 2-Dimensional (gray image) : Parameters I, II, III are used

For 2-Dimensional (color image) : Parameters I,II, III, IV are used

1 - Dimensional Convolution

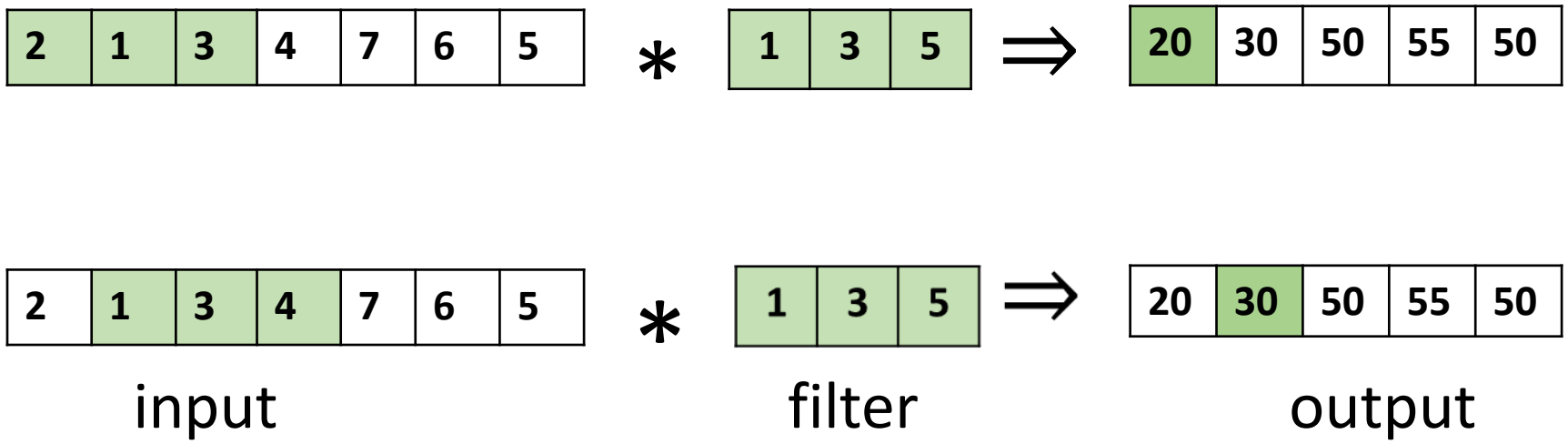
Convolution Operation : text (1 D)

Convolutional Operation	text	Example
Input size (1D)	n	7
filter size	f	3
stride	s = 1, 2, 3, …	s = 1 (default)
Output size (1D)	((n - f)/ s) + 1	o/p : (7 - 3) + 1 = 5

1-Dimensional convolution

Example

- **Input - 1D**
- **Filter size -1D**
- **Stride – 1 (default)**
- **Output- 1D**



- Sliding one dimensional filter over one dimensional input

2-Dimensional Convolution

Gray Scale Image (1 Channel)

3	0	1	2	7	4
1	5	8	9	3	1
2	7	23	50	250	3
5	55	34	3	1	89
67	45	4	56	34	23
17	13	17	20	23	16

6X6 Matrix

Each Pixel value is in the range of 0-255

- **Conversion of gray scale image into a 2-dimensional matrix of pixels**

2-Dimensional Convolution

Convolution Operation : Gray Scale image (2D)

Convolutional operation	Gray Scale image	Example
input size (2D)	n_x x n_y	6 x 6
filter size	f x f	3 x 3
stride	s = 1,2,3, …	s = 1 (default)
padding	p = 0, 1, 2,…	p = 0 (default)
output size (2D)	$(n_x + 2p - f)/s + 1$ x $(n_y + 2p - f)/s + 1$	((6+2*0-3)/1+1) x (6+2*0-3)/1+1)) o/p matrix - 4 x 4

Convolution Operation - Gray Scale Image

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6x6

Gray scale image

Convolution operator

*

1	0	-1
1	0	-1
1	0	-1

3x3

Filter

3^{1}	0^{0}	1^{-1}
1^{1}	5^{0}	8^{-1}
2^{1}	7^{0}	2^{-1}

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

4x4

output

So, we take the first 3 X 3 matrix from the 6 X 6 image and multiply it with the filter. Now, the first element of the 4 X 4 output will be the **sum of the element-wise product of these values**, i.e. 3*1 + 0 + 1*-1 + 1*1 + 5*0 + 8*-1 + 2*1 + 7*0 + 2*-1 = -5. To calculate the second element, we will **shift the filter one step towards the right** and again get the **sum of the element-wise product**

Convolution Operation - Gray Scale Image

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

→

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

Convolution Operation - Gray Scale Image

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

→

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

Convolution Operation - Gray Scale Image

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

1	0	-1
1	0	-1
1	0	-1

→

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

Convolution Operation - Gray Scale Image

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

*

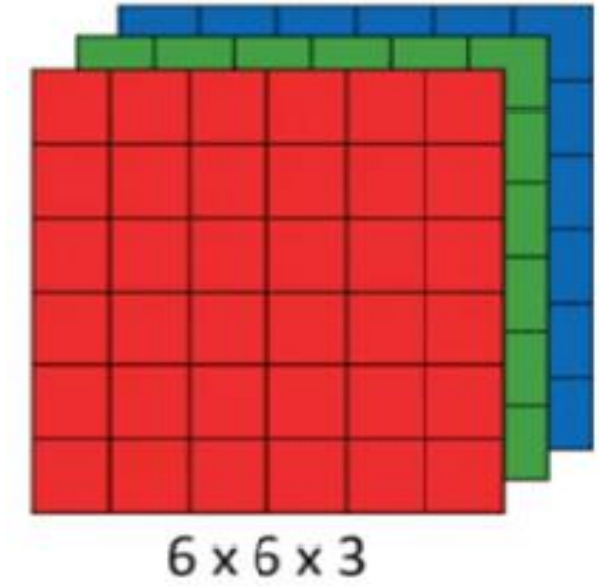
1	0	-1
1	0	-1
1	0	-1

→

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

3-Dimensional convolution

Colour Image (3 Channels)



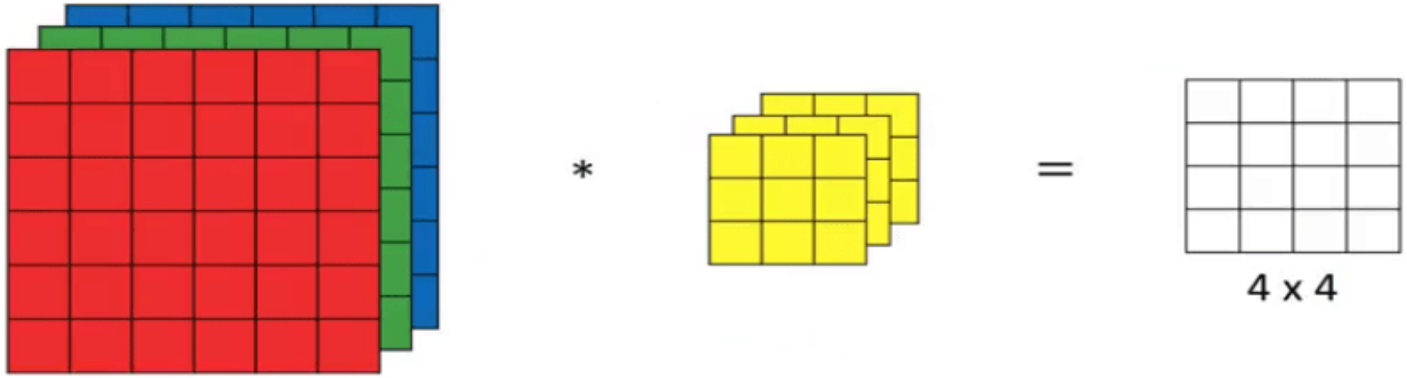
Each Pixel is in the range of 0-255

Convolution operation - Color image

input : 6 X 6 X 3 **input : 3D**

filter : 3 X 3 X 3 **filter : 3D**

output : 4 X 4 **output : 2D**



Keep in mind that the number of channels in the input and filter should be same

After convolution, the output shape is a 4 X 4 matrix

First element of the output is the sum of the element-wise product of the 27 values from the input (9 values from each channel) and the 27 values from the filter

After that we convolve over the entire image

Padding

Convolving **a 6x6 input** with a **3x3 filter** results in **4x4 output**

i.e the input size is **not retained** after convolution.

disadvantages: 1. The size of the image shrinks, after convolution operation

2. Pixels present in the **corners of the image** are used only a few no. of times as compared to central pixels - we do not focus too much on the corners, **it can lead to information loss**.

To overcome these issues, we use **padding** i.e **padding the image with an additional border**

i.e we **add one pixel** all around the edges

There are **two common choices** for padding:

- **valid:** It means **no padding i.e p=0**
- **same:** we apply **padding** so that the **output size** **is** **same** as the **input size**, i.e., **n+2p-f+1 = n, so p = (f-1)/2**

0	**0**	**0**	**0**	**0**
0	0	1	2	**0**
0	3	4	5	**0**
0	6	7	8	**0**
0	**0**	**0**	**0**	**0**

Padding

For the input matrix, we add one pixel all around the edges.

This means that the input will be an 8 x 8 matrix (instead of a 6x6 matrix)

Applying convolution of 3 x 3 on 8 x 8 will result in a 6 x 6 matrix

which is the original shape of the image

Input: n x n - **6 x 6**

Padding: p = (f-1)/2 =(3-1)/2=1

Filter size: f X f - 3 x 3

Output: (n+2p-f+1) x (n+2p-f+1) = (6+2-3+1) x (6+2-3+1) = **6 x 6**

0	**0**	**0**	**0**	**0**	**0**	**0**	**0**
0	3	0	1	2	7	4	**0**
0	1	5	8	9	3	1	**0**
0	2	7	2	5	1	3	**0**
0	0	1	3	1	7	8	**0**
0	4	2	1	6	2	8	**0**
0	2	4	5	2	3	9	**0**
0	**0**	**0**	**0**	**0**	**0**	**0**	**0**

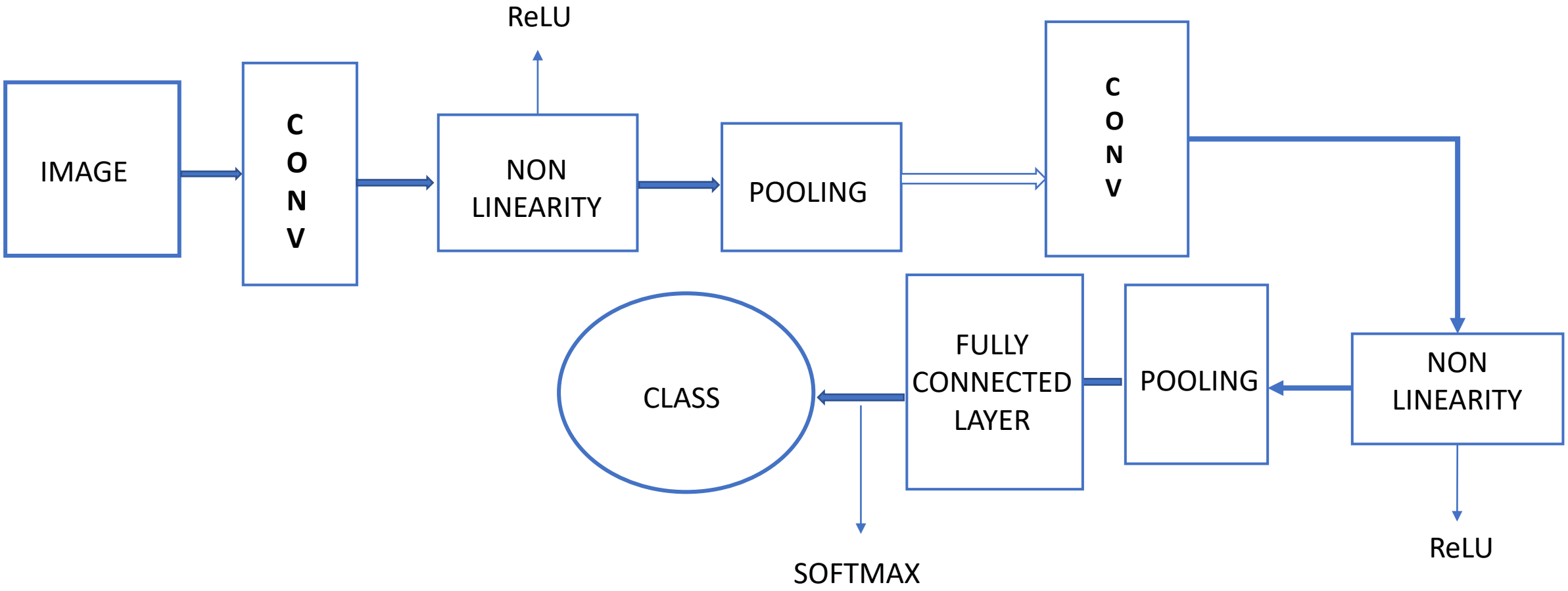
*

1	0	-1
1	0	-1
1	0	-1

→

-5	-5	-6	-1	6	10
-12	-5	-4	0	8	11
-13	-10	-2	2	3	11
-10	0	-2	-4	-7	10
-7	-3	-2	-3	-16	12
-6	0	-2	1	-9	5

CNN Architecture



*ReLU = Rectified Linear Unit

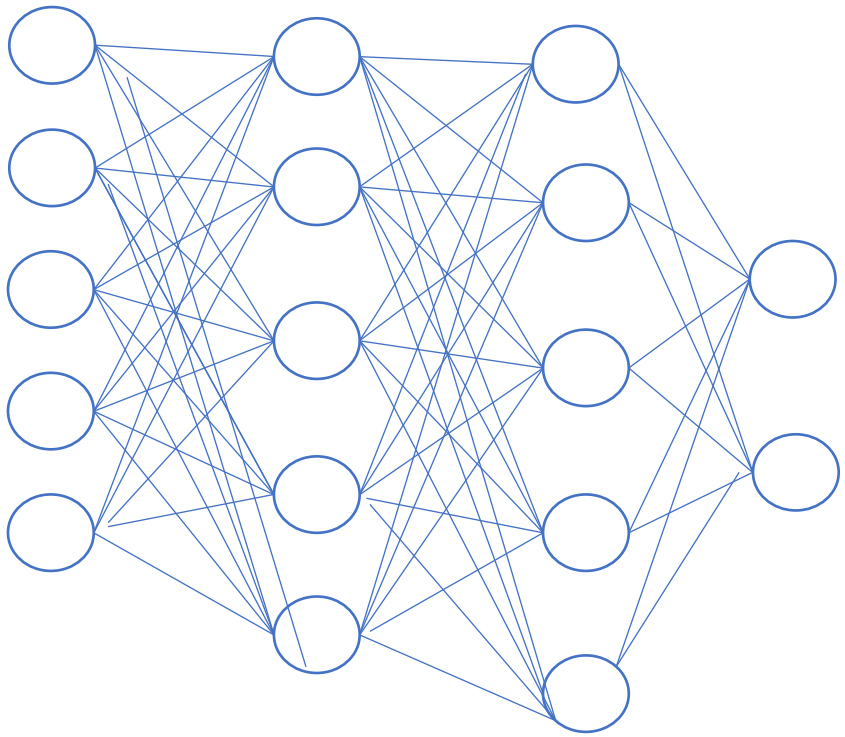
Why Convolutions?

- **Convolutions reduce the number of parameters and speed up the training of the model significantly**
- For example **14 million parameters in a fully connected layer** can be reduced to just **156 parameters in case of convolutional layer**

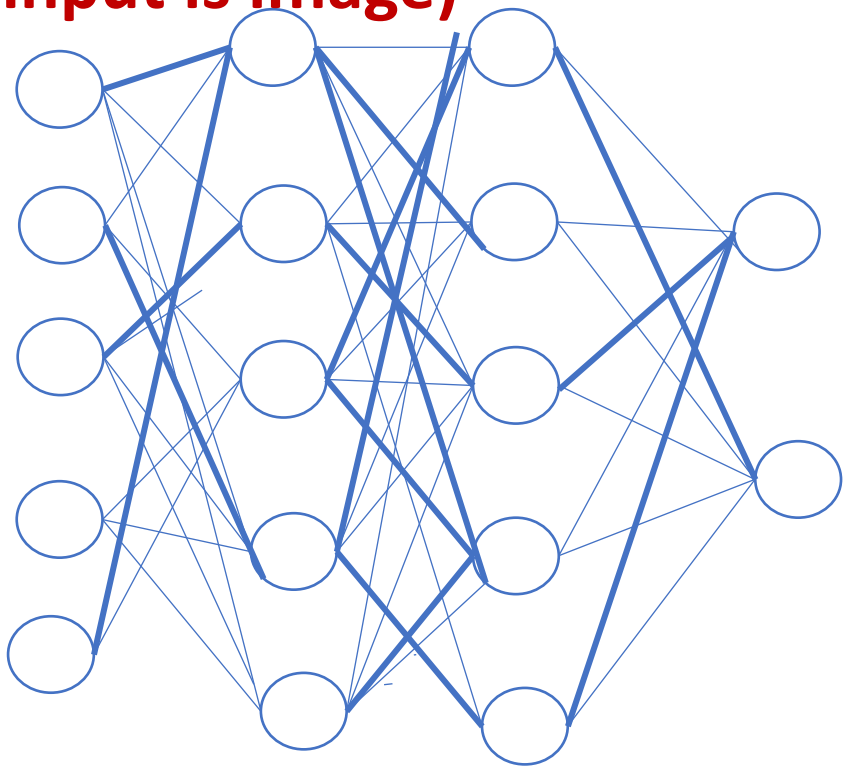
Advantages of convolutional layers over fully connected layers:

1. **Parameter sharing:** In convolutions, **a single filter is convolved over the entire input.** Due to this, **the parameters are shared between input and output nodes**
2. **Sparsity of connections:** For each layer, **each output value depends on a small number of inputs**, instead taking account all the inputs i.e the **weights of most of the connections are zeros**

ANN (input is vector)



CNN (input is image)



- **In ANN, all the layers are fully connected layers, due to this there will be millions of parameters/weights (on each connection separate weight) which increases computational complexity**

In CNN, the number of parameters/weights is independent of the size of the image. It **depends on the filter size**

Advantages of CNN:

1. **Sparsity of connections**
2. **Parameter sharing**

(Bolded connections are with 0 weights)

2. Pooling layer (Pooling operation)

Pooling operation: Reduce the **size of the input**

advantage: **speed up the computation**

Types of Pooling:

1. **Max pooling:** Maximum element in the pooling window is selected
2. **Min pooling:** Minimum element in the pooling window is selected
3. **Avg pooling:** Average of all the elements of the pooling window is selected

Generally we use max pooling or avg pooling

Hyperparameters for pooling operation:

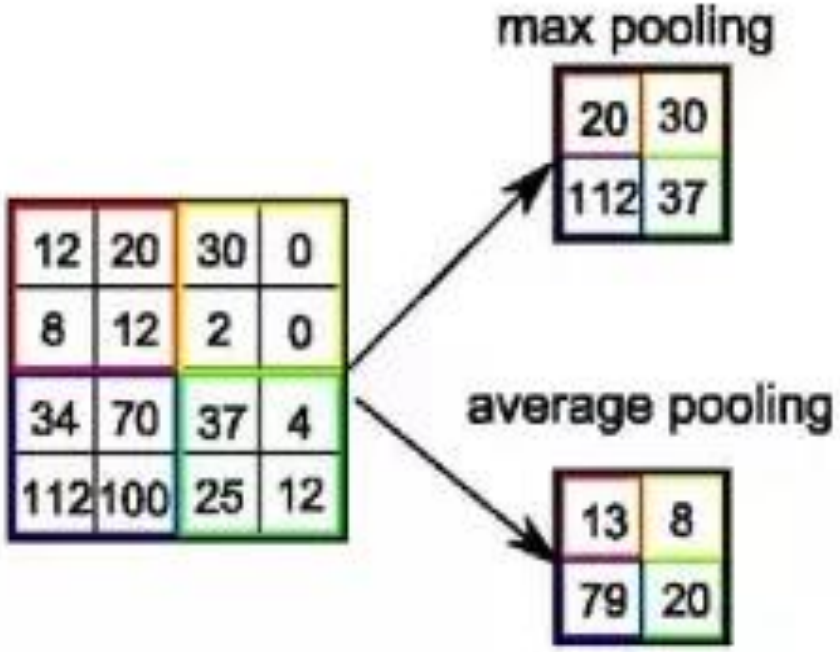
1. Filter size (elements are not required)
2. Stride
3. Max pooling or average pooling

2. Pooling layer

- The **pooling layer** will always **reduce** the size of each feature map by a **factor of 2.**

Ex 1: If size of input is **4x4** then after applying pooling operation, size of output is **2x2**

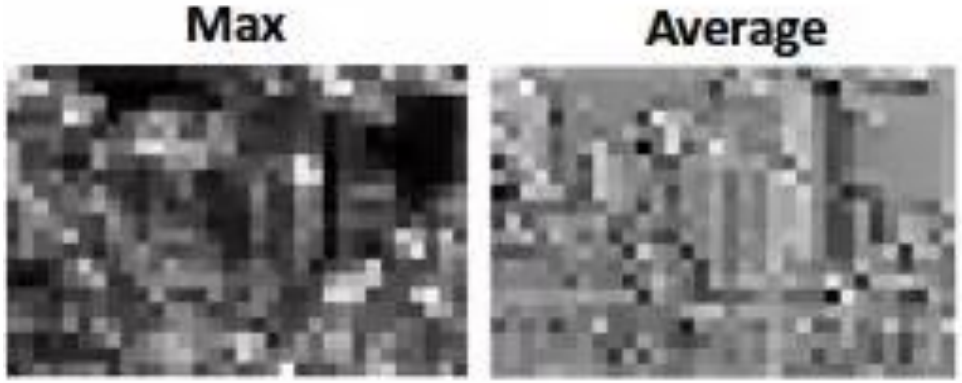
Ex 2: If size of input is **6x6** then after applying pooling operation, size of output is **3x3**



Hyperparameters:

1. Filter size -2x2
2. Stride - 2
3. Max pooling or average pooling

Note: Max/Avg pooling preserves the important features



2. Pooling Layer

-5	-5	-6	-1	6	10
-12	-5	-4	0	8	11
-13	-10	-2	2	3	11
-10	0	-2	-4	-7	10
-7	-3	-2	-3	-16	12
-6	0	-2	1	-9	5

6x6

Max pooling

-5	0	11
0	2	11
0	1	12

3x3

3. Flattening Layer

Fully connected input layer (flatten): Takes the output of the previous layer, ”flattens” it and turns it into a single vector that can be an input for the next stage.

Flattening(Unrolling): Conversion of matrix in to vector

Ex:

-5	0	11
0	2	11
0	1	12

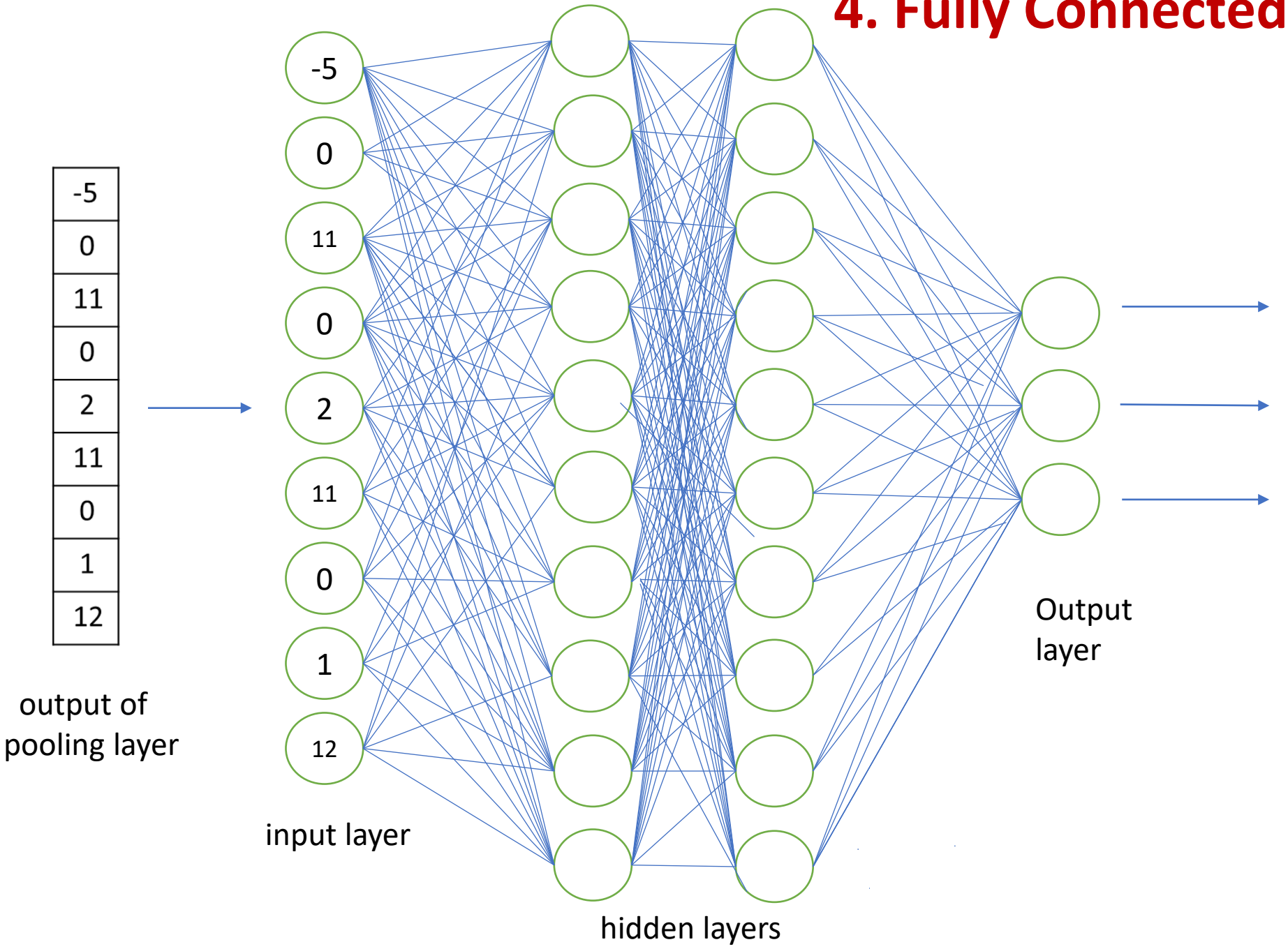
Output of the pooling layer

unrolling

-5
0
11
0
2
11
0
1
12

Input to the Fully Connected layer

4. Fully Connected Layer



Convolutional Neural Network(CNN)

Order to be followed

1. Convolutional Layer

2. Pooling Layer (optional)

3. Flattening(unrolling)

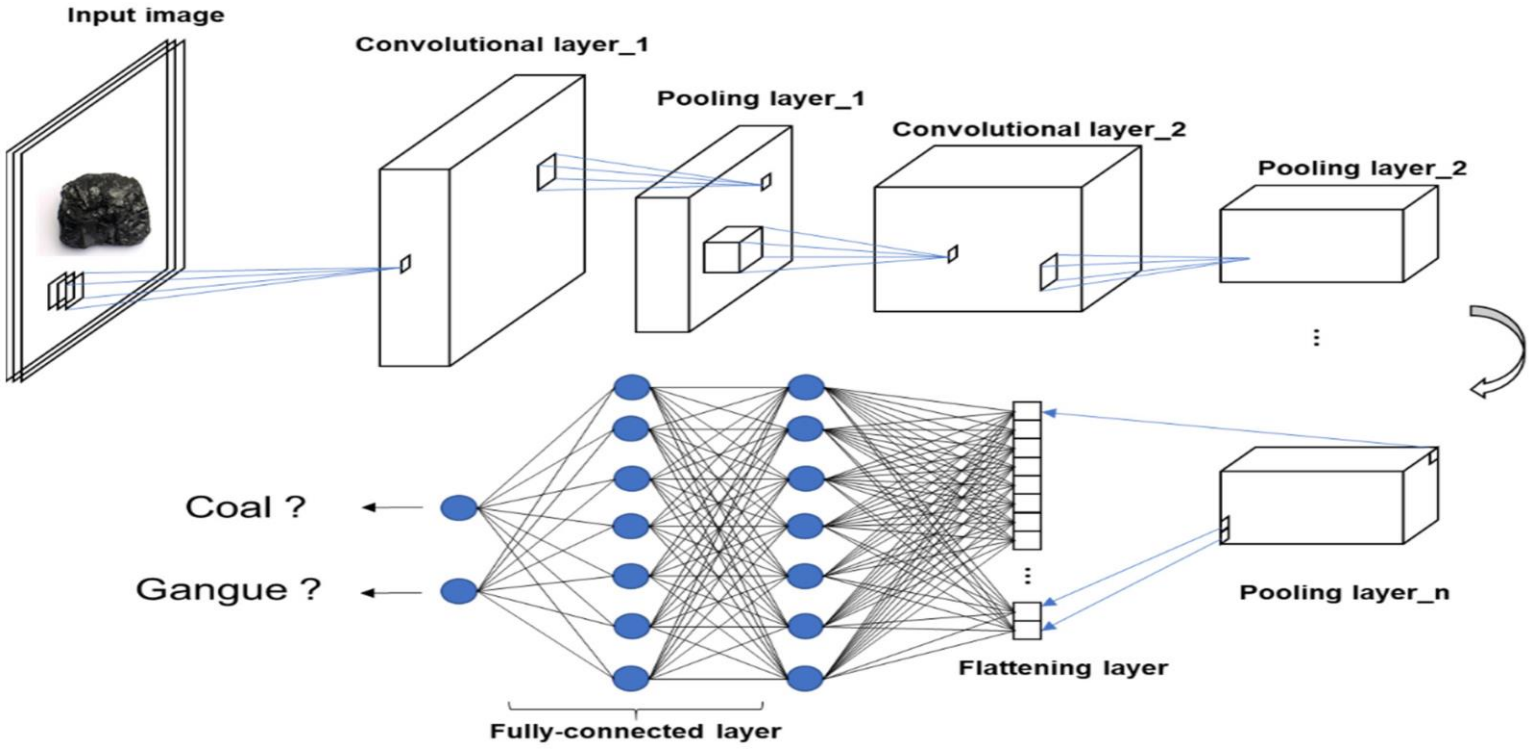
4. Fully connected layer

Note 1: The first layer must be convolutional layer

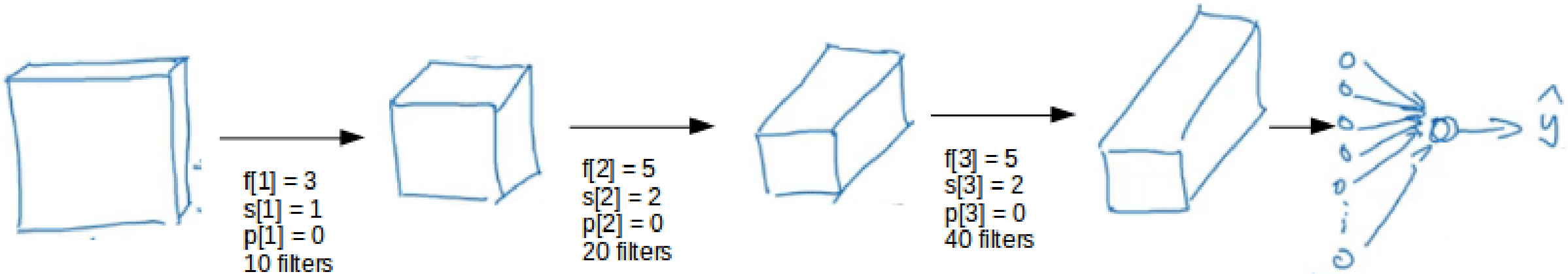
Note 2: At the end we can take any no.of fully connected layers

Convolutional Neural Network (structure)

- **The first fully connected layer** takes the inputs from the feature analysis and applies weights to predict the correct label.
- **Fully connected output layer** gives the final probabilities for each label.



Ex 1: CNN with only convolution layers



nh[0] = nw[0] = 39
nc[0] = 3

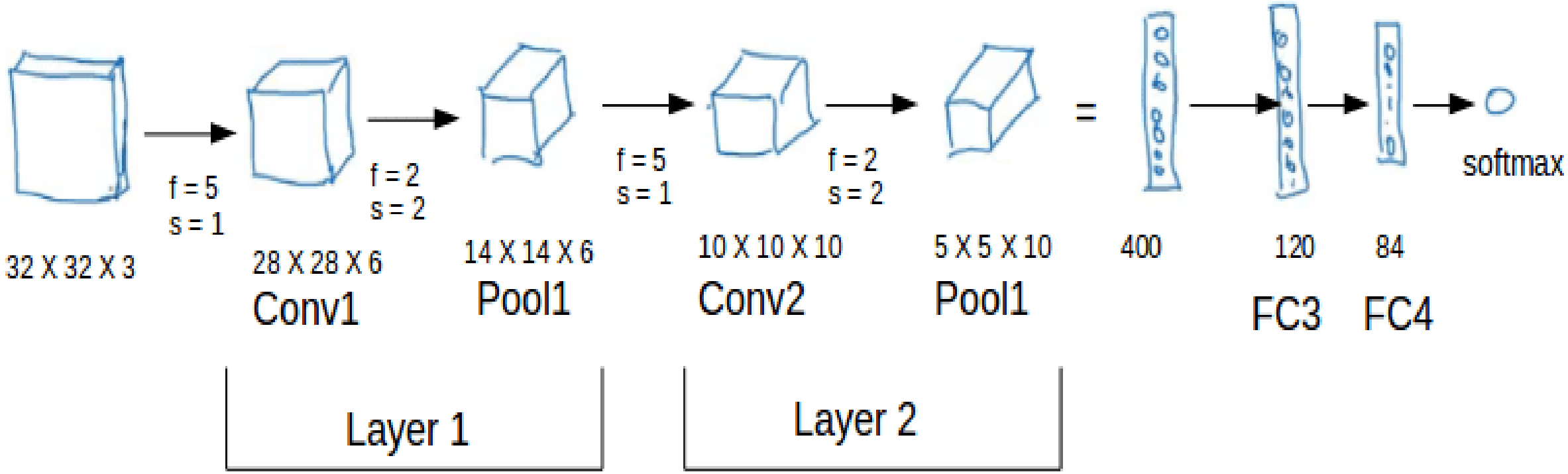
nh[1] = nw[1] = 37
nc[1] = 10

nh[2] = nw[2] = 17
nc[1] = 20

nh[1] = nw[1] = (n + 2p – f) / s + 1
nh[1] = nw[1] = (39 + 0 – 3) / 1 + 1
nh[1] = nw[1] = 37

As we go deeper into the network , input size shrinks i.e from 39x39 to 7x7 and the no. of channels increases i.e from 3 to 40

Ex 2: CNN with Convolution and Pooling layers



As we go deeper into the network , input size shrinks i.e from 32x32 to 5x5 and the no. of channels increases i.e from 3 to 10

CLASSIC CNNs

Popular networks of CNN:

1. **LeNet-5**
2. **AlexNet**
3. **VGG-16**
4. **ResNet**
5. **Inception**

Projects on CNN

1. Hand Written Digit Recognition - MNIST dataset (2D)

2. Object Recognition in Photographs - CIFAR 10/100 dataset (3D)

3. Predict Sentiment from Movie Reviews - IMDB dataset (1D)

Projects on CNN

1. Handwritten Digit Recognition - MNIST dataset (2D)

2. Object Recognition in Photographs - CIFAR 10/100 dataset (3D)

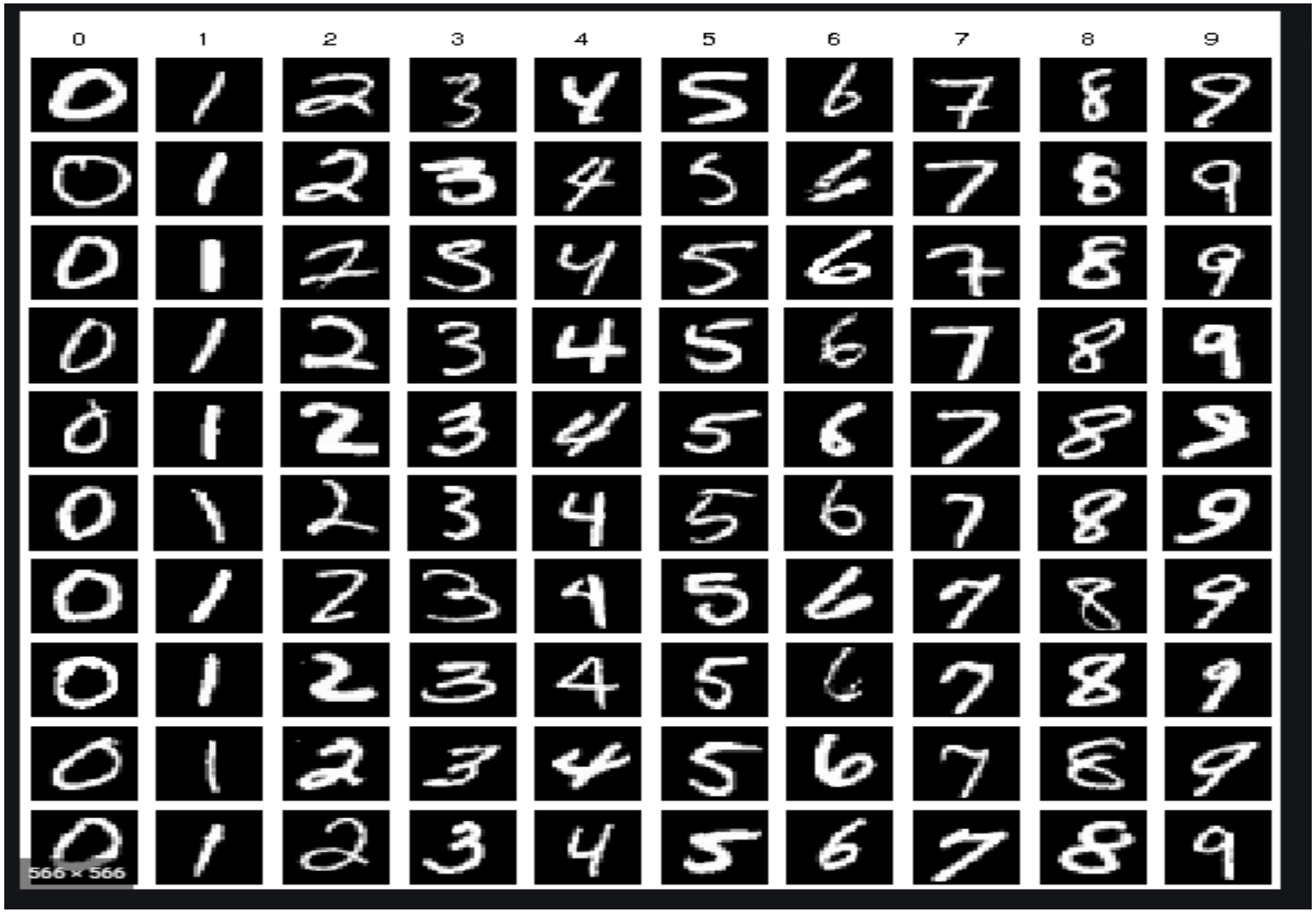
3. Predict Sentiment from Movie Reviews - IMDB dataset (1D)

1.Hand Written Digit Recognition - MNIST dataset

(Multi-class classification , gray scale images, 2D convolution)

- **MNIST dataset:** (Modified National Institute of Standards and Technology database) is a large database of **handwritten digits** that is used for training various image processing systems
- The **database** is also widely used for training and testing in the field of **machine learning and deep learning.**
- This dataset consists of **70000 images** where each image is a **handwritten digit**, out of which **60000** are for training and **10000 are for testing.**
- **70000** handwritten digits are divided into **10 classes**. Classes include digits such as **0,1,2,3,4,5,6,7,8,9**
- Size of each image is a **28x28** pixel square(**784 pixels** total)

MNIST(Sample dataset)

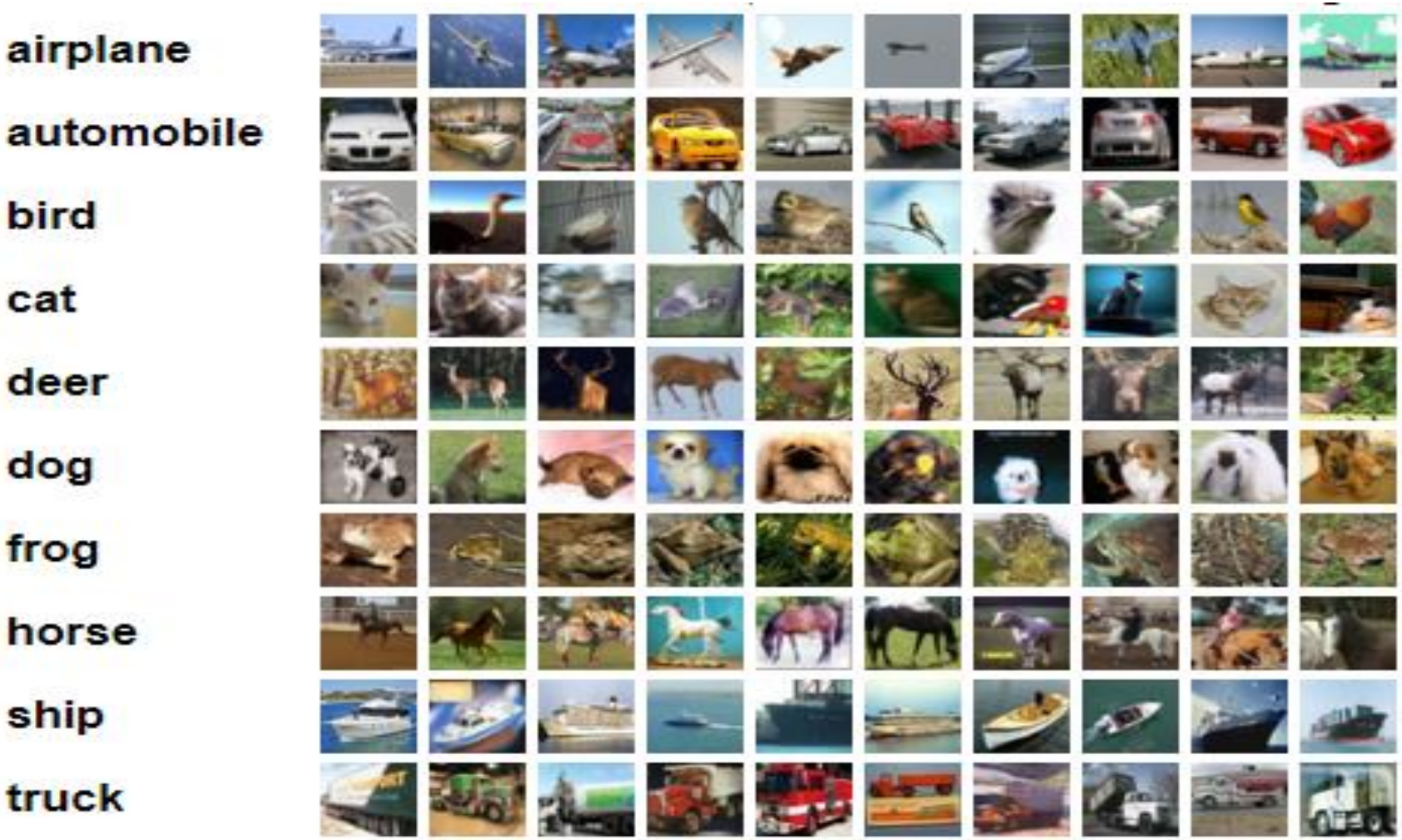


2.Object Recognition in Photographs-CIFAR 10 dataset

(multi-class classification, color images, 3D convolution)

- **CIFAR dataset:** (Canadian Institute For Advanced Research) is a large database of Photographs of objects that is used for training various image processing systems ,frog,horse
- The **CIFAR-10** consists of tiny colour images
- The **database** is also widely used for training and testing in the field of **deep learning.**
- This dataset consists of **60000 photos** where each object is a **photograph**, out of which **50000** are for training and **10000 are for testing.**
- **60000** photographs are divided into **10 classes**. Classes include objects such as airplanes, automobiles, birds, cats, frog, horse, ship, truck etc
- Size of each image is a **32x32** pixel squares (**784 pixels** total)

CIFAR10 (Sample Dataset)



3. Predict Sentiment from Movie Reviews-IMDB dataset

(Binary classification, 1D convolution)

- The Internet Movie Database (IMDB) is a huge repository for image and text data.
- The **database** is an excellent source for data analytics and deep learning practice and research
- This is a dataset for binary sentiment classification consists of 50,000

highly polar movie reviews.

- **50000 movie reviews** are divided into **2 classes**. Classes include positive reviews and negative reviews
- This is a dataset for with 25,000 highly polar movie reviews for training, and 25,000 for testing.

IMDB (Sample Dataset)

S.No	Review	Sentiment
1	This movie features Charlie Spradling dancing in a strip club. Beyond that, it features a truly bad script with dull, unrealistic dialogue. That it got as many positive votes suggests some people may be joking.	negative
2	If you like original gut wrenching laughter you will like this movie. If you are young or old then you will love this movie, hell even my mom liked it. Great Camp!!!	Positive
3	It's terrific when a funny movie doesn't make smile you. What a pity!! This film is very boring and so long. It's simply painfull. The story is staggering without goal and no fun. You feel better when it's finished.	negative
4	I have seen this film at least 100 times and I am still excited by it, the acting is perfect and the romance between Joe and Jean keeps me on the edge of my seat, plus I still think Bryan Brown is the tops. Brilliant Film.	positive

Datasets - MNIST / CIFAR10 / IMDB

S.No	dataset	type of dataset	No. of classes	class labels	classification	convolution
1	MNIST	Handwritten digits (gray scale images)	10	0,1,2,3,4,5,6,7,8,9	multiclass	2D
2	CIFAR10	images of objects (Color images)	10	airplane, automobile, dog, deer, cat, truck, ship, horse, bird, frog	multiclass	3D
3	IMDB	movie reviews	2	positive, negative	binary	1D

Steps to develop Convolutional Neural Networks(CNN) with Keras

Step 1: Load Data

Step 2: Define Model

Step 3: Compile Model

Step 4: Fit Model

Step 5: Evaluate Model

Hand Written Digit Recognition

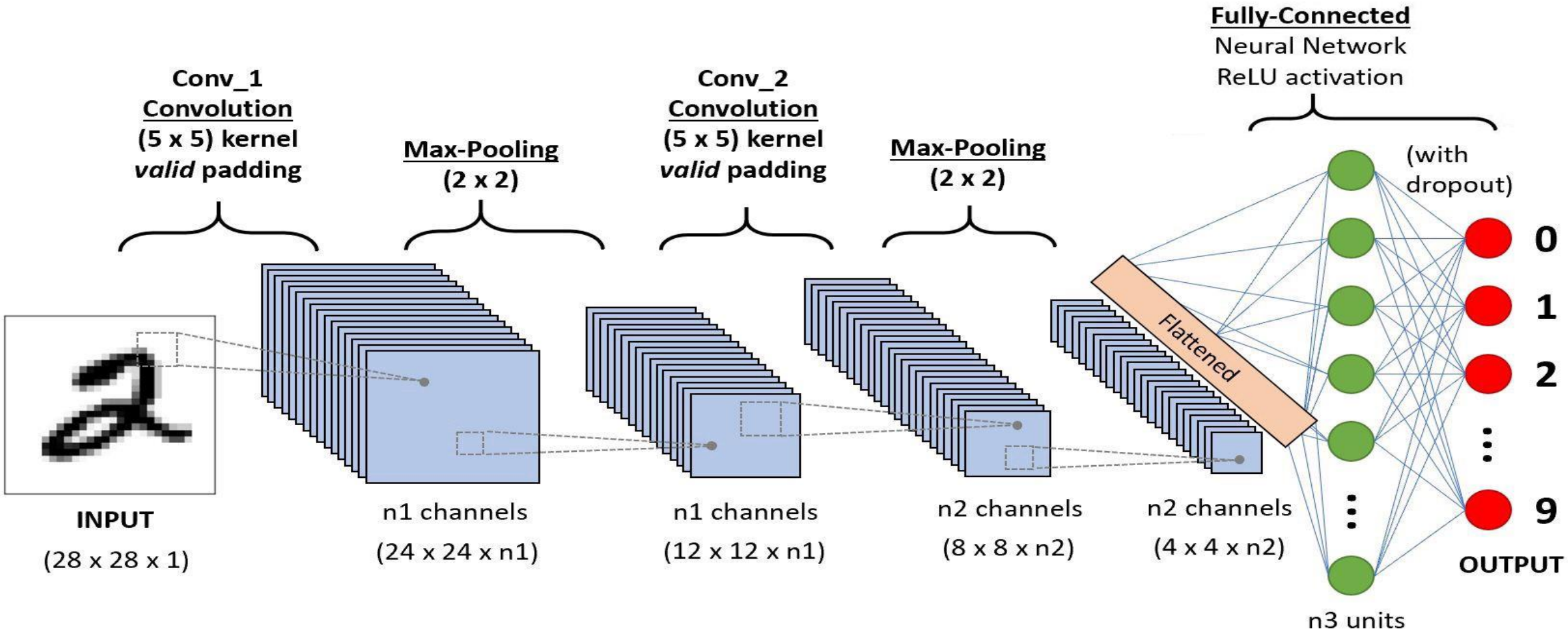


Image augmentation

- Image augmentation generates random images based on existing training data to improve the generalization ability of models.

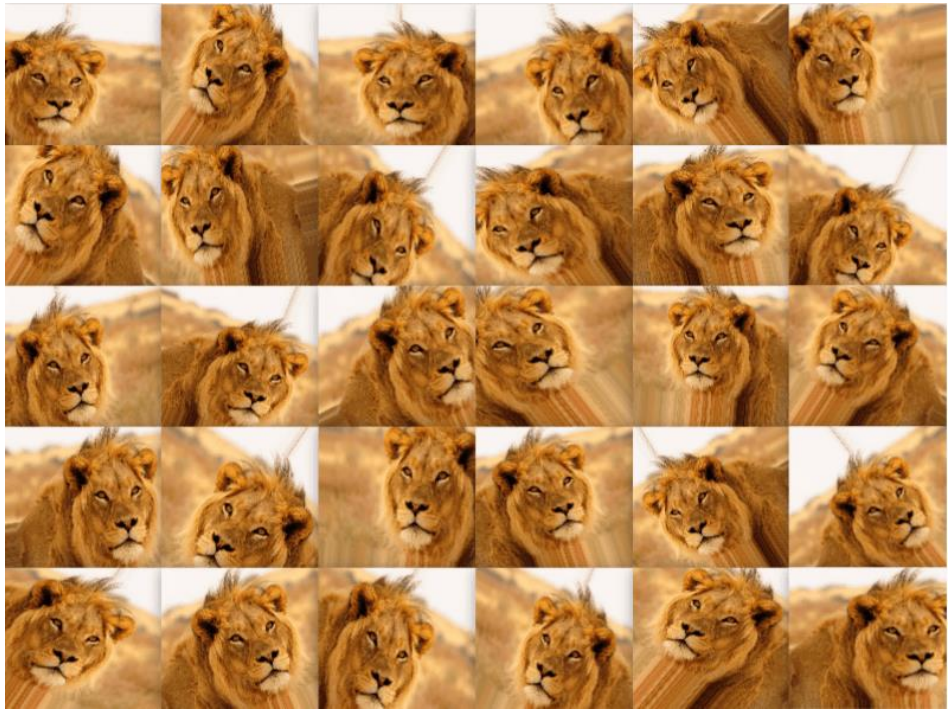


Image augmentation

- Image augmentation is a technique that is used to artificially expand the data-set.
- This is helpful when we are given a data-set with very few data samples.
- In case of Deep Learning, this situation is bad as the model tends to over-fit when we train it on limited number of data samples.
- Image augmentation parameters that are generally used to increase the data sample count are zoom, shear, rotation, preprocessing_function and so on.
- Usage of these parameters results in generation of images having these attributes during training of Deep Learning model.
- Image samples generated using image augmentation, in general results in increase of existing data sample set by nearly 3x to 4x times.